

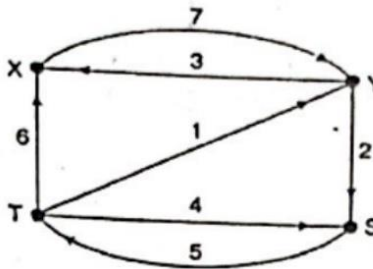
SEGMENT: 05(GRAPH)

Previous Problem Solve

Spring: 2024

5.

a) Use Warshall's algorithm to find the path matrix for the graph G in following figure. Show each step.



Solve:

Applying the modified Warshall's algorithm,

$D_0 =$

	X	Y	T	S
X	0	7	∞	∞
Y	3	0	∞	2
T	6	1	0	4
S	∞	∞	5	0

$D_1 =$

	X	Y	T	S
X	0	7	∞	∞
Y	3	0	∞	2
T	6	1	0	4
S	∞	∞	5	0

$D_2 =$

	X	Y	T	S
X	0	7	∞	9
Y	3	0	∞	2
T	4	1	0	4
S	∞	∞	5	0

Update:

$$D_2[X][S] = \min(\infty, D_1[X][Y] + D_1[Y][S]) = 9$$

$$D_2[T][X] = \min(\infty, D_1[T][Y] + D_1[Y][X]) = 4$$

$D_3 =$

	X	Y	T	S
X	0	7	∞	9
Y	3	0	∞	2
T	4	1	0	4
S	11	6	5	0

Update:

$$D_3[S][X] = \min(\infty, D_2[S][T] + D_2[T][X]) = 11$$

$$D_3[S][Y] = \min(\infty, D_2[S][T] + D_2[T][Y]) = 6$$

$D_4 =$

	X	Y	T	S
X	0	7	14	9
Y	3	0	7	2
T	4	1	0	4
S	9	6	5	0

Update:

$$D_4[X][T] = \min(\infty, D_3[X][S] + D_3[S][T]) = 14$$

$$D_4[Y][T] = \min(\infty, D_2[Y][S] + D_3[S][T]) = 7$$

b) Define graph? Draw a picture of the directed graph specified below:

$G = (V, E)$

$V(G) = \{1, 2, 3, 4, 5, 6\}$

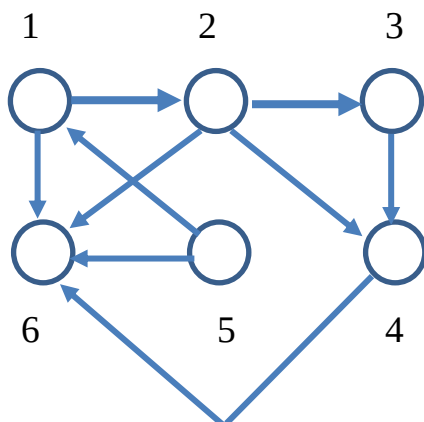
$E(G) = \{(1,2), (2,3), (3,4), (5,1), (5,6), (2,6), (1,6), (4,6), (2,4)\}$

Obtain the following for the above graph:

- Find the adjacency matrix A of the graph G .
- Find the adjacency list of the graph G .

Solve:

A graph is a mathematical structure used to model relationships between objects.

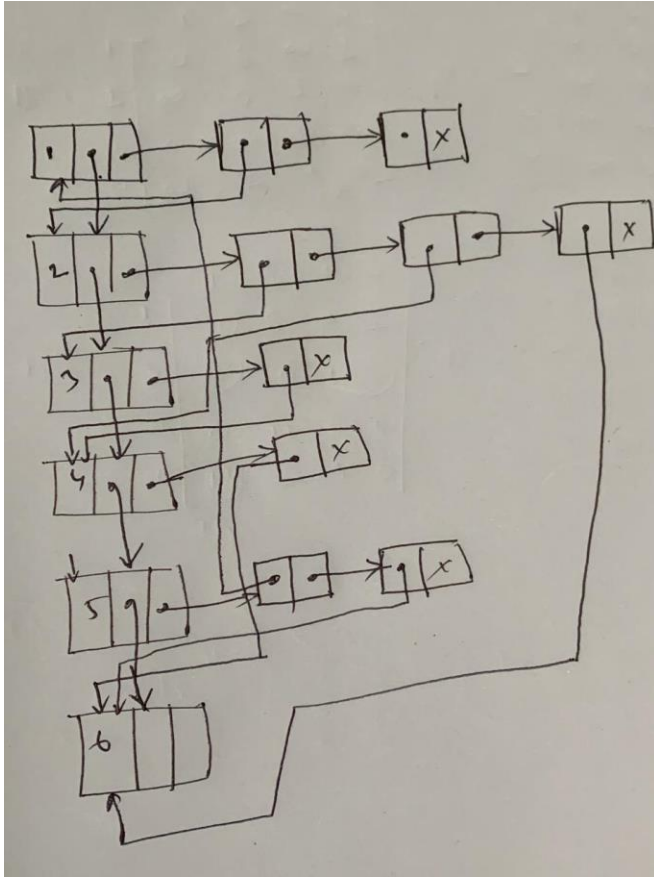


i) Adjacency matrix A of the graph G

	1	2	3	4	5	6
1	0	1	0	0	0	1
2	0	0	1	1	0	1
3	0	0	0	1	0	0
4	0	0	0	0	0	1
5	1	0	0	0	0	1
6	0	0	0	0	0	0

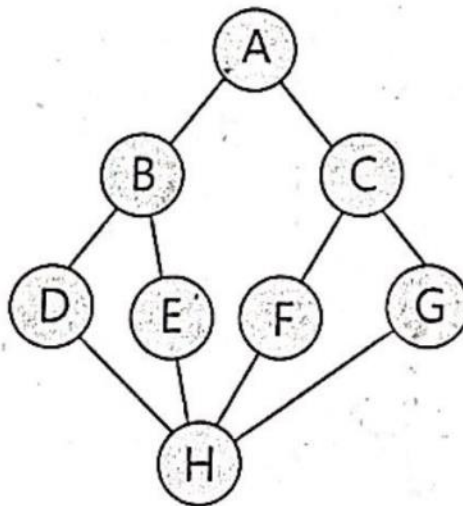
ii) Adjacency list:

Node	Adjancency List
1	2, 6
2	3, 4, 6
3	4
4	6
5	1, 6
6	



OR

b) Traverse the graph G shown below in breadth first order, depth first order and construct the breadth first and depth first spanning trees. Start from node D [if last digit of your ID is odd]/ G [if last digit of your ID is even].



Breadth-First Search (BFS) starting at D

Graph Adjacency:

- **A**: Neighbors $\rightarrow$ B, C
- **B**: Neighbors $\rightarrow$ A, D, E
- **C**: Neighbors $\rightarrow$ A, F, G
- **D**: Neighbors $\rightarrow$ B, H
- **E**: Neighbors $\rightarrow$ B, F
- **F**: Neighbors $\rightarrow$ C, E, H
- **G**: Neighbors $\rightarrow$ C, H
- **H**: Neighbors $\rightarrow$ D, F, G

Initial Setup:

- Start BFS from **D**.
- Add **D** to the QUEUE.
- Visited: [D]
QUEUE: [D]
ORIG: [$\emptyset$] (no origin for starting node)

Step 1:

- Remove **D** from the front of the queue.
- Add its unvisited neighbors: **B** and **H**.
- Update origins: **B** $\rightarrow$ **D**, **H** $\rightarrow$ **D**.
- Visited: [D, B, H]
QUEUE: [B, H]
ORIG: [$\emptyset$, D, D]

Step 2:

- Remove **B** from the front of the queue.
- Add its unvisited neighbors: **A** and **E**.
- Update origins: **A** $\rightarrow$ **B**, **E** $\rightarrow$ **B**.
- Visited: [D, B, H, A, E]
QUEUE: [H, A, E]
ORIG: [$\emptyset$, D, D, B, B]

Step 3:

- Remove **H** from the front of the queue.
- Add its unvisited neighbors: **F** and **G**.
- Update origins: **F** $\rightarrow$ **H**, **G** $\rightarrow$ **H**.

- Visited: [D, B, H, A, E, F, G]
 QUEUE: [A, E, F, G]
 ORIG: [$\emptyset$, D, D, B, B, H, H]

Step 4:

- Remove **A** from the front of the queue.
- Add its unvisited neighbor: **C**.
- Update origins: **C** $\rightarrow$ **A**.
- Visited: [D, B, H, A, E, F, G, C]
 QUEUE: [E, F, G, C]
 ORIG: [$\emptyset$, D, D, B, B, H, H, A]

Step 5:

- Remove **E** from the front of the queue.
- **E**'s neighbors (**B**, **F**) are already visited. Proceed.
- Visited: [D, B, H, A, E, F, G, C]
 QUEUE: [F, G, C]
 ORIG: [$\emptyset$, D, D, B, B, H, H, A]

Step 6:

- Remove **F** from the front of the queue.
- **F**'s neighbors (**C**, **E**, **H**) are already visited. Proceed.
- Visited: [D, B, H, A, E, F, G, C]
 QUEUE: [G, C]
 ORIG: [$\emptyset$, D, D, B, B, H, H, A]

Step 7:

- Remove **G** from the front of the queue.
- **G**'s neighbors (**C**, **H**) are already visited. Proceed.
- Visited: [D, B, H, A, E, F, G, C]
 QUEUE: [C]
 ORIG: [$\emptyset$, D, D, B, B, H, H, A]

Step 8:

- Remove **C** from the front of the queue.
- **C**'s neighbors (**A**, **F**, **G**) are already visited. Stop.
- Visited: [D, B, H, A, E, F, G, C]
 QUEUE: []
 ORIG: [$\emptyset$, D, D, B, B, H, H, A]

Final BFS Traversal Order:

D → B → H → A → E → F → G → C